

PRACTICAL TEST AUTOMATION WITH PLAYWRIGHT

Playwright Setup Guide

Get a clean, working Playwright project running from scratch.

Introduction

Before you can automate anything, you need a reliable setup. This guide takes you from nothing to a running Playwright project with TypeScript, so the rest of the course (and your portfolio) has a solid base.

Prerequisites

- Node.js (LTS) installed
- A code editor (VS Code recommended)
- Basic command line familiarity

Step-by-Step

1. Create a project folder and open a terminal there.
2. Run: `npm init playwright@latest`
3. Choose TypeScript, a tests folder, and add a GitHub Actions workflow when prompted.
4. Run the example tests: `npx playwright test`
5. Open the HTML report: `npx playwright show-report`

Worked Example (project layout)

Path	Purpose
playwright.config.ts	Global config (browsers, baseURL, reporters)
tests/	Your test files (*.spec.ts)
tests-examples/	Generated example tests
playwright-report/	HTML test report output

Verify It Works

- `npx playwright test` runs and passes the example tests
- `npx playwright test --ui` opens the UI mode
- `npx playwright codegen <url>` records actions

Practical Exercise

Set up a fresh project, run the example tests, then use codegen against a public site (e.g. a demo shop) and save the generated test.

Best Practices

- Use TypeScript from day one.
- Commit your project to GitHub early (great for your portfolio).
- Keep dependencies updated.

Common Mistakes

- Skipping the example tests, so setup issues hide until later.
- Not installing browsers (`npx playwright install`).
- Committing `node_modules` instead of using `.gitignore`.

INSIDE STLC PRO TIP

Push your Playwright project to GitHub from the very first commit. A public automation repo with a clear README is one of the strongest portfolio pieces a tester can show.